AI workflow note

Tools used

- **Claude (Opus 5.5, in the Claude app with a code workspace)** as a pair programmer. It scaffolded the app, wrote most of the first-pass code, ran the tests and a headless-browser walkthrough, and drafted these notes. I set the scope, chose the stack and sharing model, and reviewed each stage.
- **Walkthrough video.** Claude wrote the narration script from these notes and screen-recorded the walkthrough with Playwright, against a local production build of the same code that's deployed. The voiceover is an open-source text-to-speech voice (Kokoro, "Heart"), not my own voice.

Where AI materially sped things up

- **Scaffolding and boilerplate:** the Next.js setup, Drizzle schema and migrations, API routes with validation, and the seed script took minutes instead of an hour.
- **Library details:** TipTap v3 APIs (`useEditorState`, server-side `generateJSON`), mammoth for .docx, and Drizzle's conditional updates.
- **Verification at scale:** a curl smoke test of every permission and error path, plus a scripted headless-Chrome run through create, format, reload, share, view as another user, import, and a two-user save conflict, with screenshots checked after each UI change.

AI output that was changed or rejected

These happened during the session:

- **ORM swap.** The first plan used Prisma. Its engine binaries couldn't be downloaded in the build environment, and Prisma also adds cold-start weight on serverless. Switched to Drizzle (pure JavaScript, plain SQL migrations).
- **Autosave retry logic.** The first version of the editor's save loop read stale React state when deciding whether to retry, so it could retry forever after a server error. Rewritten so that it only auto-saves again after a *successful* save that had more edits queued. After a failure it waits for a manual retry or the next edit.
- **Conflict test that passed for the wrong reason.** The first smoke test of "stale save is rejected" sent an empty document, which failed schema validation, not the version check. Caught by reading the response, then re-tested with valid content to confirm the real 409.

- **Conflict UX, rejected twice.** After a conflict the editor first stayed editable, so a user could keep typing text that would never save. The next version locked it and asked for a reload, but reloading threw away the user's recent edits. The final version loads the latest copy in place and keeps the user's edits beside it.
- **Link insertion bug found in browser testing.** Typing right after adding a link lost the first keystrokes and kept extending the link. The cause: TipTap's `focus()` waits a frame, so keys went to the popover's button. Fixed by focusing synchronously and ending the link mark after inserting it.
- **Missing server-side link check.** The schema validation accepted a `javascript:` link sent straight to the API, because it checks structure, not attribute values. Added explicit link validation, with tests.
- **Toolbar overflow.** Adding alignment, quote and link buttons wrapped the toolbar onto two lines, so those controls became icons with accessible labels.
- **UI fixes from screenshot review:** the toolbar wasn't aligned with the page, the "Text" button lit up inside lists, the topbar didn't fit on mobile, and the import hint copy was awkward.
- **Fonts.** The default `next/font/google` setup fetches fonts at build time, and the build environment blocked that. Switched to self-hosted `@fontsource` packages, which also removes a runtime dependency on Google.
- **A wrong diagnosis during deployment.** After the first deploy, the obvious URL (`ajaia-assessment.vercel.app`) returned 500s, and the AI's first theory was that Vercel had picked Node 26. It pinned Node 22 and redeployed, but the 500 persisted. The request logs showed the real cause: our requests never reached the project at all, because that URL belongs to someone else's Vercel project. Ours is `ajaia-assessment-kappa.vercel.app`. The Node pin and a Postgres pool error handler were kept as sensible hardening, but they weren't the fix. The lesson: check where the request actually went before changing code.

How correctness, UX and reliability were verified

- **Automated tests (63):** the permission matrix, file import (including script stripping and rejection of bad, empty and oversized files), schema validation, Markdown export with an import/export round trip, link safety, and Postgres integration tests for demo-account creation, sharing, role changes, revoking access, and stale-save conflicts.
- **End-to-end checks:** the live Vercel deployment was exercised in a real browser (sign in, create, format, reload, share as viewer, open as the viewer). Before that, curl against a local production build for auth (including a forged cookie), every role boundary, validation errors, and import. Headless Chrome for the full user flows on desktop and mobile widths, including two signed-in users editing the same document at once. The only console output was the browser logging the expected 409 during the conflict test.
- **Not taking AI output at face value:** every test was read for *what* it asserts, not just whether it passes (see the conflict-test example above).

My review

I used Claude as the main implementer and kept the product and scope decisions for myself.

- **Scope and stack.** I chose the stack (Next.js, TipTap, Postgres on Neon) and the working split: Claude writes the code in stages, and I review each stage and make the calls.
- **Asking for criticism, then prioritizing.** Once the first version worked, I asked for an honest 1–10 rating instead of a summary. From the weaknesses it listed, I picked three to fix: the conflict handling losing user edits, a thin editor toolbar, and no export. I left real-time co-editing and real authentication out on purpose, because they couldn't be done well in the timebox.
- **Keeping the infrastructure minimal.** Neon's setup offered extra services (object storage, auth, functions). I kept it to Postgres only, because the app never stores files, and an unused bucket would be one more thing to explain and secure.
- **Secrets stayed with me.** Claude set up the Vercel project and the environment variable names, but I entered the database connection string and the session secret myself and completed Vercel's identity check.
- **What I read closely:** [fill in: for example, `src/lib/access.ts` (the permission rules), `updateDocument` in `src/lib/documents.ts` (the version check), and the save loop in `src/components/DocEditor.tsx`, plus anything you'd change or found unclear].
- **What I tested by hand on the live site:** [fill in after you click through it yourself, for example: shared a doc as a viewer, opened it in a second browser as an editor, triggered a save conflict].